

Engineering Case Studies: Software Failure and Resilience

Lessons from Apollo 11, Ariane 5, and the AT&T Network Collapse

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

To analyse landmark failures in complex software systems, understanding their root causes, the technical mechanisms of failure, and the enduring lessons for modern software engineering. We will examine the critical importance of designing for resilience, the risks of flawed assumptions in redundancy and testing, and the impact of subtle coding errors at scale.

Lecture Overview

- 1 Introduction: Learning from Failure
- 2 Case Study C1.1: Apollo 11 Program Alarms
- 3 Case Study C1.2: Ariane 5 Flight 501
- 4 Case Study C2: The 1990 AT&T Network Collapse
- 5 Conclusion

Introduction: Learning from Failure

Introduction: Learning from Failure

The Importance of Case Studies

Why Study Failure?

- Complex systems fail in complex ways. Understanding historical failures helps us anticipate and prevent future ones.
- Highlights the gap between theoretical design and real-world operation.
- Emphasises that software does not exist in isolation; it interacts with hardware, environment, and human operators.

The Cases

We will examine three distinct types of failure:

- 1 **Apollo 11:** Resilience in the face of unforeseen hardware/software interaction.
- 2 **Ariane 5:** Catastrophic failure due to flawed assumptions and software reuse.
- 3 **AT&T Network Collapse:** Cascading failure caused by a single line of code and inadequate testing.

The Importance of Case Studies

Why Study Failure?

- Complex systems fail in complex ways. Understanding historical failures helps us anticipate and prevent future ones.
- Highlights the gap between theoretical design and real-world operation.
- Emphasises that software does not exist in isolation; it interacts with hardware, environment, and human operators.

The Cases

We will examine three distinct types of failure:

- 1 **Apollo 11:** Resilience in the face of unforeseen hardware/software interaction.
- 2 **Ariane 5:** Catastrophic failure due to flawed assumptions and software reuse.
- 3 **AT&T Network Collapse:** Cascading failure caused by a single line of code and inadequate testing.

Case Study C1.1: Apollo 11 Program Alarms

Case Study C1.1: Apollo 11 Program Alarms

Apollo 11: The Lunar Descent

The Context: July 20, 1969

The Lunar Module (Eagle) is descending to the Moon's surface. Neil Armstrong and Buzz Aldrin are relying entirely on Apollo Guidance Computer (AGC) for navigation and control.

The Event: 1202 Program Alarm

During the critical final minutes of descent, the AGC begins issuing unexpected alarms: 1202 and later 1201.

- Neither astronaut had encountered these alarms in training.
- The computer was failing thousands of metres above the Moon.

The Decision

Should they abort the landing?

Apollo 11: The Lunar Descent

The Context: July 20, 1969

The Lunar Module (Eagle) is descending to the Moon's surface. Neil Armstrong and Buzz Aldrin are relying entirely on Apollo Guidance Computer (AGC) for navigation and control.

The Event: 1202 Program Alarm

During the critical final minutes of descent, the AGC begins issuing unexpected alarms: 1202 and later 1201.

- Neither astronaut had encountered these alarms in training.
- The computer was failing thousands of metres above the Moon.

The Decision

Should they abort the landing?

Apollo 11: The Lunar Descent

The Context: July 20, 1969

The Lunar Module (Eagle) is descending to the Moon's surface. Neil Armstrong and Buzz Aldrin are relying entirely on Apollo Guidance Computer (AGC) for navigation and control.

The Event: 1202 Program Alarm

During the critical final minutes of descent, the AGC begins issuing unexpected alarms: 1202 and later 1201.

- Neither astronaut had encountered these alarms in training.
- The computer was failing thousands of metres above the Moon.

The Decision

Should they abort the landing?

The Hardware: Apollo Guidance Computer (AGC)

To understand the alarms, we must understand the constraints of the AGC.

A Marvel of its Time

- One of the first computers to use integrated circuits.
- Designed for extreme constraints on size, weight, and power.

Severe Memory Limitations

ROM (Read-Only Memory): 72 Kilobytes (Core Rope Memory).

RAM (Random Access Memory): 4 Kilobytes (Magnetic Core Memory).

The Hardware: Apollo Guidance Computer (AGC)

To understand the alarms, we must understand the constraints of the AGC.

A Marvel of its Time

- One of the first computers to use integrated circuits.
- Designed for extreme constraints on size, weight, and power.

Severe Memory Limitations

ROM (Read-Only Memory): 72 Kilobytes (Core Rope Memory).

RAM (Random Access Memory): 4 Kilobytes (Magnetic Core Memory).

The Hardware: Apollo Guidance Computer (AGC)

A Marvel of its Time

- One of the first computers to use integrated circuits.
- Designed for extreme constraints on size, weight, and power.

Severe Memory Limitations

ROM (Read-Only Memory): 72 Kilobytes (Core Rope Memory).

RAM (Random Access Memory): 4 Kilobytes (Magnetic Core Memory).

The Constraint

This single computer managed engines, navigation, radar, life support, and cockpit displays simultaneously.

The Software Design: Executive Control

Task Management

The software ran multiple tasks by rapidly rotating between them (cooperative multitasking).

Memory Management: Core Sets

The 4KB of RAM was insufficient to hold all potential tasks simultaneously.

- Memory was divided into small chunks called **Core Sets** (24 bytes each).
- Tasks requested core sets as needed.
- The system relied on careful planning to ensure enough core sets were always available for the current flight phase.

The Software Design: Executive Control

Task Management

The software ran multiple tasks by rapidly rotating between them (cooperative multitasking).

Memory Management: Core Sets

The 4KB of RAM was insufficient to hold all potential tasks simultaneously.

- Memory was divided into small chunks called **Core Sets** (24 bytes each).
- Tasks requested core sets as needed.
- The system relied on careful planning to ensure enough core sets were always available for the current flight phase.

The Software Design: Executive Control

The Meaning of the Alarms

- **1201 Alarm:** Executive Overflow (No more core sets available).
- **1202 Alarm:** Executive Overflow (No more vacancy areas).

The computer had run out of memory to perform its computations.

The Root Cause: An Unforeseen Interaction

Why did the computer run out of memory during the landing phase?

The Rendezvous Radar

- The radar used for docking with the Command Module (not needed for landing).
- A last-minute decision was made to leave it powered on "just in case" an abort was needed.

The Hardware Flaw

Unknown to the team, a design fault in the power system caused the rendezvous radar's timing mechanism to be **out of phase** with the computer's CPU clock.

The Root Cause: An Unforeseen Interaction

Why did the computer run out of memory during the landing phase?

The Rendezvous Radar

- The radar used for docking with the Command Module (not needed for landing).
- A last-minute decision was made to leave it powered on "just in case" an abort was needed.

The Hardware Flaw

Unknown to the team, a design fault in the power system caused the rendezvous radar's timing mechanism to be **out of phase** with the computer's CPU clock.

The Root Cause: An Unforeseen Interaction

The Rendezvous Radar

- The radar used for docking with the Command Module (not needed for landing).
- A last-minute decision was made to leave it powered on "just in case" an abort was needed.

The Hardware Flaw

Unknown to the team, a design fault in the power system caused the rendezvous radar's timing mechanism to be **out of phase** with the computer's CPU clock.

The Effect: Cycle Stealing

This misalignment caused the radar to constantly generate spurious signals (glitches). Handling these glitches "stole" small amounts of CPU time ($\approx 15\%$ of the CPU capacity).

The Failure Mechanism

The Cascade

- 1 The CPU load increased due to the radar glitches.
- 2 Tasks were not completing as quickly as planned.
- 3 As tasks fell behind, they requested more core sets to store intermediate data.
- 4 Eventually, the available core sets were exhausted.
- 5 The 1201/1202 alarms were triggered.

The Computer's Response to Memory Exhaustion

When the AGC detected this critical overflow, it did the only thing it could do: **It rebooted.**

The Failure Mechanism

The Cascade

- 1 The CPU load increased due to the radar glitches.
- 2 Tasks were not completing as quickly as planned.
- 3 As tasks fell behind, they requested more core sets to store intermediate data.
- 4 Eventually, the available core sets were exhausted.
- 5 The 1201/1202 alarms were triggered.

The Computer's Response to Memory Exhaustion

When the AGC detected this critical overflow, it did the only thing it could do: **It rebooted.**

The Failure Mechanism

The Cascade

- 1 The CPU load increased due to the radar glitches.
- 2 Tasks were not completing as quickly as planned.
- 3 As tasks fell behind, they requested more core sets to store intermediate data.
- 4 Eventually, the available core sets were exhausted.
- 5 The 1201/1202 alarms were triggered.

The Computer's Response to Memory Exhaustion

When the AGC detected this critical overflow, it did the only thing it could do: **It rebooted.**

The Failure Mechanism

The Cascade

- 1 The CPU load increased due to the radar glitches.
- 2 Tasks were not completing as quickly as planned.
- 3 As tasks fell behind, they requested more core sets to store intermediate data.
- 4 Eventually, the available core sets were exhausted.
- 5 The 1201/1202 alarms were triggered.

The Computer's Response to Memory Exhaustion

When the AGC detected this critical overflow, it did the only thing it could do: **It rebooted.**

The Failure Mechanism

The Cascade

- 1 The CPU load increased due to the radar glitches.
- 2 Tasks were not completing as quickly as planned.
- 3 As tasks fell behind, they requested more core sets to store intermediate data.
- 4 Eventually, the available core sets were exhausted.
- 5 The 1201/1202 alarms were triggered.

The Computer's Response to Memory Exhaustion

When the AGC detected this critical overflow, it did the only thing it could do: **It rebooted.**

The Failure Mechanism

The Cascade

- 1 The CPU load increased due to the radar glitches.
- 2 Tasks were not completing as quickly as planned.
- 3 As tasks fell behind, they requested more core sets to store intermediate data.
- 4 Eventually, the available core sets were exhausted.
- 5 The 1201/1202 alarms were triggered.

The Computer's Response to Memory Exhaustion

When the AGC detected this critical overflow, it did the only thing it could do: **It rebooted.**

The Saving Grace: Designing for Resilience

Why didn't the reboot cause a catastrophic loss of control?

The Foresight of Margaret Hamilton

Late in development, Margaret Hamilton (leader of the software team) insisted on a critical design change, fearing a potential crash during descent.

Restartable Tasks

All critical software tasks (navigation, guidance, engine control) were modified to be **restartable**.

The Saving Grace: Designing for Resilience

Why didn't the reboot cause a catastrophic loss of control?

The Foresight of Margaret Hamilton

Late in development, Margaret Hamilton (leader of the software team) insisted on a critical design change, fearing a potential crash during descent.

Restartable Tasks

All critical software tasks (navigation, guidance, engine control) were modified to be **restartable**.

The Saving Grace: Designing for Resilience

The Recovery Mechanism

When the computer rebooted:

- 1 It cleared out all non-essential tasks, freeing up the core sets.
- 2 It immediately restarted the highest priority critical tasks.
- 3 Control was maintained before the spacecraft could deviate significantly.

The Human Factor and the Lesson

The Decision in Mission Control

Engineer Jack Garman, who studied every possible error code, made critical assessment.

- He concluded the landing was safe **as long as the alarms were not too frequent**.
- Mission control gave the "Go" command, despite the computer rebooting repeatedly.

The Lesson Learned

The mission was saved not by bug-free software, but by software designed to be **resilient** to unforeseen events. The engineers anticipated failure and designed a robust recovery mechanism.

Further Reading

Sunburst and Luminary by Don Eyles (who wrote the landing phase software).

The Human Factor and the Lesson

The Decision in Mission Control

Engineer Jack Garman, who studied every possible error code, made critical assessment.

- He concluded the landing was safe **as long as the alarms were not too frequent**.
- Mission control gave the "Go" command, despite the computer rebooting repeatedly.

The Lesson Learned

The mission was saved not by bug-free software, but by software designed to be **resilient** to unforeseen events. The engineers anticipated failure and designed a robust recovery mechanism.

Further Reading

Sunburst and Luminary by Don Eyles (who wrote the landing phase software).

The Human Factor and the Lesson

The Decision in Mission Control

Engineer Jack Garman, who studied every possible error code, made critical assessment.

- He concluded the landing was safe **as long as the alarms were not too frequent**.
- Mission control gave the "Go" command, despite the computer rebooting repeatedly.

The Lesson Learned

The mission was saved not by bug-free software, but by software designed to be **resilient** to unforeseen events. The engineers anticipated failure and designed a robust recovery mechanism.

Further Reading

Sunburst and Luminary by Don Eyles (who wrote the landing phase software).

Case Study C1.2: Ariane 5 Flight 501

Case Study C1.2: Ariane 5 Flight 501

Ariane 5: A Catastrophic Failure

The Context: June 4, 1996

The maiden flight of the European Space Agency's Ariane 5 heavy-lift booster, launching from French Guiana.

The Event

- 37 seconds after launch, the rocket suddenly commanded a 90-degree turn.
- The intense aerodynamic stress caused the booster to break apart.
- The automatic self-destruct mechanism activated.

The Impact

One of the most expensive software bugs in history. The failure highlights critical flaws in assumptions about software reuse and redundancy.

Ariane 5: A Catastrophic Failure

The Context: June 4, 1996

The maiden flight of the European Space Agency's Ariane 5 heavy-lift booster, launching from French Guiana.

The Event

- 37 seconds after launch, the rocket suddenly commanded a 90-degree turn.
- The intense aerodynamic stress caused the booster to break apart.
- The automatic self-destruct mechanism activated.

The Impact

One of the most expensive software bugs in history. The failure highlights critical flaws in assumptions about software reuse and redundancy.

Ariane 5: A Catastrophic Failure

The Context: June 4, 1996

The maiden flight of the European Space Agency's Ariane 5 heavy-lift booster, launching from French Guiana.

The Event

- 37 seconds after launch, the rocket suddenly commanded a 90-degree turn.
- The intense aerodynamic stress caused the booster to break apart.
- The automatic self-destruct mechanism activated.

The Impact

One of the most expensive software bugs in history. The failure highlights critical flaws in assumptions about software reuse and redundancy.

The System Architecture

The Inertial Reference System (SRI)

The SRI is responsible for measuring the rocket's position, velocity, and orientation.

Redundancy

Given its critical role, the system used two identical SRIs (SRI 1 and SRI 2).

- If the primary SRI (SRI 1) failed, the system would immediately failover to the backup (SRI 2).

The Onboard Computer (OBC)

The active SRI feeds data to the OBC, which uses it to keep the rocket on course by controlling the engine nozzles.

The System Architecture

The Inertial Reference System (SRI)

The SRI is responsible for measuring the rocket's position, velocity, and orientation.

Redundancy

Given its critical role, the system used two identical SRIs (SRI 1 and SRI 2).

- If the primary SRI (SRI 1) failed, the system would immediately failover to the backup (SRI 2).

The Onboard Computer (OBC)

The active SRI feeds data to the OBC, which uses it to keep the rocket on course by controlling the engine nozzles.

The System Architecture

The Inertial Reference System (SRI)

The SRI is responsible for measuring the rocket's position, velocity, and orientation.

Redundancy

Given its critical role, the system used two identical SRIs (SRI 1 and SRI 2).

- If the primary SRI (SRI 1) failed, the system would immediately failover to the backup (SRI 2).

The Onboard Computer (OBC)

The active SRI feeds data to the OBC, which uses it to keep the rocket on course by controlling the engine nozzles.

The Root Cause: Software Reuse and Assumptions

The Strategy: Reusing Proven Code

The software routine responsible for the failure was reused directly from the Ariane 5's predecessor, the Ariane 4.

The Flawed Assumption

The assumption was that reusing "proven" and reliable code was safer than developing and testing new software.

The Fatal Difference

The Ariane 5 was significantly faster and accelerated more quickly than the Ariane 4. The software was operating outside the physical regime for which it had been tested.

The Root Cause: Software Reuse and Assumptions

The Strategy: Reusing Proven Code

The software routine responsible for the failure was reused directly from the Ariane 5's predecessor, the Ariane 4.

The Flawed Assumption

The assumption was that reusing "proven" and reliable code was safer than developing and testing new software.

The Fatal Difference

The Ariane 5 was significantly faster and accelerated more quickly than the Ariane 4. The software was operating outside the physical regime for which it had been tested.

The Root Cause: Software Reuse and Assumptions

The Strategy: Reusing Proven Code

The software routine responsible for the failure was reused directly from the Ariane 5's predecessor, the Ariane 4.

The Flawed Assumption

The assumption was that reusing "proven" and reliable code was safer than developing and testing new software.

The Fatal Difference

The Ariane 5 was significantly faster and accelerated more quickly than the Ariane 4. The software was operating outside the physical regime for which it had been tested.

The Technical Flaw: Integer Overflow

The Calculation: Horizontal Bias (BH)

The SRI software calculated a value related to the horizontal velocity, known as Horizontal Bias (BH).

The Data Types

- 1 The calculation was performed using high-precision **64-bit floating-point** numbers.
- 2 The result was then stored in a **16-bit signed integer**.

The Overflow

37 seconds into the flight, the horizontal velocity of the Ariane 5 became too large to fit into a 16-bit integer (Maximum value: 32,767).

The Technical Flaw: Integer Overflow

The Calculation: Horizontal Bias (BH)

The SRI software calculated a value related to the horizontal velocity, known as Horizontal Bias (BH).

The Data Types

- 1 The calculation was performed using high-precision **64-bit floating-point** numbers.
- 2 The result was then stored in a **16-bit signed integer**.

The Overflow

37 seconds into the flight, the horizontal velocity of the Ariane 5 became too large to fit into a 16-bit integer (Maximum value: 32,767).

The Technical Flaw: Integer Overflow

The Calculation: Horizontal Bias (BH)

The SRI software calculated a value related to the horizontal velocity, known as Horizontal Bias (BH).

The Data Types

- 1 The calculation was performed using high-precision **64-bit floating-point** numbers.
- 2 The result was then stored in a **16-bit signed integer**.

The Overflow

37 seconds into the flight, the horizontal velocity of the Ariane 5 became too large to fit into a 16-bit integer (Maximum value: 32,767).

The Technical Flaw: Integer Overflow

The Data Types

- 1 The calculation was performed using high-precision **64-bit floating-point** numbers.
- 2 The result was then stored in a **16-bit signed integer**.

The Overflow

37 seconds into the flight, the horizontal velocity of the Ariane 5 became too large to fit into a 16-bit integer (Maximum value: 32,767).

The Exception

This caused an **unprotected overflow exception** (Operand Error) in the Ada programming language.

The Cascading Failure: Step by Step

- 1 **(T+37s)** The overflow error occurred on the primary SRI (SRI 1). The software declared itself failed and shut down.
- 2 The system automatically failed over to the redundant SRI (SRI 2).
- 3 **(T+37.072s)** SRI 2 was running the **exact same software**. It experienced the **exact same overflow** 72 milliseconds later and also failed.

The Flaw in Redundancy

The redundancy design only accounted for **random hardware failures**, not a **systematic software failure** that would affect both identical units simultaneously.

The Cascading Failure: Step by Step

- 1 **(T+37s)** The overflow error occurred on the primary SRI (SRI 1). The software declared itself failed and shut down.
- 2 The system automatically failed over to the redundant SRI (SRI 2).
- 3 **(T+37.072s)** SRI 2 was running the **exact same software**. It experienced the **exact same overflow** 72 milliseconds later and also failed.

The Flaw in Redundancy

The redundancy design only accounted for **random hardware failures**, not a **systematic software failure** that would affect both identical units simultaneously.

The Cascading Failure: Step by Step

- 1 **(T+37s)** The overflow error occurred on the primary SRI (SRI 1). The software declared itself failed and shut down.
- 2 The system automatically failed over to the redundant SRI (SRI 2).
- 3 **(T+37.072s)** SRI 2 was running the **exact same software**. It experienced the **exact same overflow** 72 milliseconds later and also failed.

The Flaw in Redundancy

The redundancy design only accounted for **random hardware failures**, not a **systematic software failure** that would affect both identical units simultaneously.

The Cascading Failure: Step by Step

- 1 **(T+37s)** The overflow error occurred on the primary SRI (SRI 1). The software declared itself failed and shut down.
- 2 The system automatically failed over to the redundant SRI (SRI 2).
- 3 **(T+37.072s)** SRI 2 was running the **exact same software**. It experienced the **exact same overflow** 72 milliseconds later and also failed.

The Flaw in Redundancy

The redundancy design only accounted for **random hardware failures**, not a **systematic software failure** that would affect both identical units simultaneously.

The Final Blow: Misinterpretation

Even with both SRIs failed, the disaster might have been averted if not for the final step.

The Diagnostic Output

When SRI 2 failed, it output a diagnostic bit pattern to the Onboard Computer (OBC).

The Fatal Misinterpretation

The OBC misinterpreted this diagnostic error code as actual flight orientation data.

The Catastrophic Command

- Based on this "data," the OBC believed the rocket was 90 degrees off course.
- It commanded full nozzle deflection on the main engine and solid rocket boosters to "correct" the trajectory.
- The rocket broke apart due to aerodynamic stress.

The Final Blow: Misinterpretation

Even with both SRIs failed, the disaster might have been averted if not for the final step.

The Diagnostic Output

When SRI 2 failed, it output a diagnostic bit pattern to the Onboard Computer (OBC).

The Fatal Misinterpretation

The OBC misinterpreted this diagnostic error code as actual flight orientation data.

The Catastrophic Command

- Based on this "data," the OBC believed the rocket was 90 degrees off course.
- It commanded full nozzle deflection on the main engine and solid rocket boosters to "correct" the trajectory.
- The rocket broke apart due to aerodynamic stress.

The Final Blow: Misinterpretation

The Diagnostic Output

When SRI 2 failed, it output a diagnostic bit pattern to the Onboard Computer (OBC).

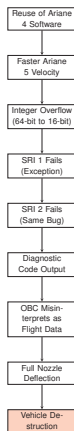
The Fatal Misinterpretation

The OBC misinterpreted this diagnostic error code as actual flight orientation data.

The Catastrophic Command

- Based on this "data," the OBC believed the rocket was 90 degrees off course.
- It commanded full nozzle deflection on the main engine and solid rocket boosters to "correct" the trajectory.
- The rocket broke apart due to aerodynamic stress.

Visualising the Failure Chain



The Lessons of Ariane 5

The Irony

The entire software routine that caused crash was part of Ariane 4's pre-launch initialisation sequence. It **was not needed at all** on the Ariane 5 after launch, yet it was kept running.

The Lessons of Ariane 5

Key Lessons

- 1 Reusing software outside the specific conditions for which it was designed and tested is extremely dangerous. "Proven" does not mean "universally safe".
- 2 Integer overflows are a common and critical vulnerability. Uncaught exceptions are often as bad as silent errors.
- 3 Redundancy must account for systematic software failures, not just random hardware failures. (Requires diversity in implementation).
- 4 Failure modes should be safe (e.g., degraded accuracy mode instead of immediate shutdown and diagnostic output).

Case Study C2: The 1990 AT&T Network Collapse

Case Study C2: The 1990 AT&T Network Collapse

The Day the Network Died

The Context: January 15, 1990

AT&T operated the largest long-distance telephone network in the US, handling ≈ 115 million calls per day. The network relied on 114 computer-controlled 4ESS switches.

The Event: A Cascading Failure

- 1 A single 4ESS switch in Manhattan suffered a minor glitch and rebooted (4 seconds).
- 2 Immediately, all its neighbouring switches crashed.
- 3 When those neighbours rebooted, *their* neighbours crashed.
- 4 The failure rippled across the entire 114-switch network.

The Day the Network Died

The Context: January 15, 1990

AT&T operated the largest long-distance telephone network in the US, handling ≈ 115 million calls per day. The network relied on 114 computer-controlled 4ESS switches.

The Event: A Cascading Failure

- 1 A single 4ESS switch in Manhattan suffered a minor glitch and rebooted (4 seconds).
- 2 Immediately, all its neighbouring switches crashed.
- 3 When those neighbours rebooted, *their* neighbours crashed.
- 4 The failure rippled across the entire 114-switch network.

The Day the Network Died

The Event: A Cascading Failure

- 1 A single 4ESS switch in Manhattan suffered a minor glitch and rebooted (4 seconds).
- 2 Immediately, all its neighbouring switches crashed.
- 3 When those neighbours rebooted, *their* neighbours crashed.
- 4 The failure rippled across the entire 114-switch network.

The Impact

The network was effectively down for nine hours. 70 million phone calls failed, causing immense economic damage (e.g., 500+ cancelled flights).

The Mechanism: Switch Communication

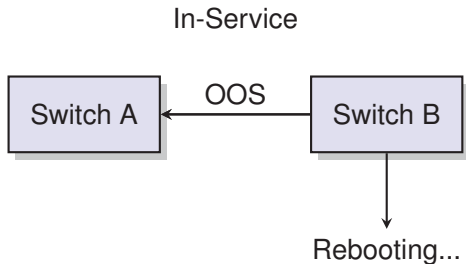
We must understand how switches handle reboots to ensure calls are not sent to a switch that is down.

The Original Protocol

- 1 Switch B goes down, sending "Out of Service" (OOS) to Switch A.
- 2 Switch B reboots.
- 3 Switch B sends an explicit "In-Service" message to Switch A.
- 4 Switch A resumes sending calls to Switch B.

The Mechanism: Switch Communication

We must understand how switches handle reboots to ensure calls are not sent to a switch that is down.



The "Optimization" and the Bug

The Optimization

AT&T engineers realised the explicit "In-Service" message was redundant.

- When the rebooted Switch B sent its first *new call* to Switch A, Switch A could use that call as an implicit signal that Switch B was back online.
- They removed the dedicated "In-Service" message.

The Flaw

The bug was introduced in the new code implemented to handle this "implicit" in-service logic. The bug resided within a 'switch' statement in the C programming language.

The "Optimization" and the Bug

The Optimization

AT&T engineers realised the explicit "In-Service" message was redundant.

- When the rebooted Switch B sent its first *new call* to Switch A, Switch A could use that call as an implicit signal that Switch B was back online.
- They removed the dedicated "In-Service" message.

The Flaw

The bug was introduced in the new code implemented to handle this "implicit" in-service logic. The bug resided within a 'switch' statement in the C programming language.

The Failure Scenario: A Race Condition

The bug only manifested under specific, high-load conditions.

The Scenario

- 1 Switch B reboots. It is very busy and immediately sends two new calls (Msg 1 and Msg 2) to Switch A in rapid succession (milliseconds apart).
- 2 Switch A still believes Switch B is "Out of Service".

The Logic Requirement

When Switch A receives a message from B (which it thinks is OOS), it needs to mark B as "In-Service". Crucially, it must ensure it doesn't process this status change twice if two messages arrive nearly simultaneously.

The Failure Scenario: A Race Condition

The bug only manifested under specific, high-load conditions.

The Scenario

- 1 Switch B reboots. It is very busy and immediately sends two new calls (Msg 1 and Msg 2) to Switch A in rapid succession (milliseconds apart).
- 2 Switch A still believes Switch B is "Out of Service".

The Logic Requirement

When Switch A receives a message from B (which it thinks is OOS), it needs to mark B as "In-Service". Crucially, it must ensure it doesn't process this status change twice if two messages arrive nearly simultaneously.

The Code: The Misplaced 'break' I

The following pseudo-code illustrates the logic in Switch A when receiving a message.

Listing 1: Simplified 4ESS Switch Code (C/Pseudo-code)

```
1 while (messages_exist) {  
2     // ... initialize pointers ...  
3     switch (message_type) {  
4         case INCOMING_CALL:  
5             if (sender_status == OUT_OF_SERVICE) {  
6                 // Check if we are already processing an in-service  
7                     message  
8                 if (in_service_task_queued == TRUE) {  
9                     break; // FATAL FLAW: Exit the 'if'?  
6                 }  
7             }  
8         }  
9     }
```

The Code: The Misplaced 'break' II

```
10         // ... queue the in-service task ...
11     }
12     // Process the incoming call
13     process_call();
14     setup_data_pointers(); // Crucial step
15     break; // Exit the switch statement
16 }
17 use_data_pointers(); // Use the pointers set up earlier
18 }
```

The Failure Mechanism: C Semantics

The Programmer's Intent

The programmer used 'break;' (Line 8) intending to exit the inner 'if' statement, preventing a duplicate "in-service" task when the second message (Msg 2) arrived.

The Reality of C Language Semantics

In C, 'break;' does not exit an 'if' statement. It exits the nearest enclosing loop or 'switch' statement.

The Consequence

The 'break;' caused the code to jump out of the **entire 'switch' statement**.

- 1 It skipped processing the incoming call (Msg 2).
- 2 Crucially, it skipped the `setup_data_pointers()` function (Line 14).

The Failure Mechanism: C Semantics

The Programmer's Intent

The programmer used 'break;' (Line 8) intending to exit the inner 'if' statement, preventing a duplicate "in-service" task when the second message (Msg 2) arrived.

The Reality of C Language Semantics

In C, 'break;' does not exit an 'if' statement. It exits the nearest enclosing loop or 'switch' statement.

The Consequence

The 'break;' caused the code to jump out of the **entire 'switch' statement**.

- 1 It skipped processing the incoming call (Msg 2).
- 2 Crucially, it skipped the `setup_data_pointers()` function (Line 14).

The Failure Mechanism: C Semantics

The Reality of C Language Semantics

In C, 'break;' does not exit an 'if' statement. It exits the nearest enclosing loop or 'switch' statement.

The Consequence

The 'break;' caused the code to jump out of the **entire 'switch' statement**.

- 1 It skipped processing the incoming call (Msg 2).
- 2 Crucially, it skipped the `setup_data_pointers()` function (Line 14).
- 3 The code then proceeded to `use_data_pointers()` (Line 17).

The Crash and the Cascade

The Crash

Since the data pointers were never initialised, the switch tried to read data from a null or corrupted memory address. This caused Switch A to crash and reboot.

The Cascade

Switch A, now crashed, rebooted and immediately sent *its* own flood of new calls (because the network was busy) to *its* neighbours. This triggered the exact same race condition and the exact same bug in them, propagating the failure across the entire network.

The Crash and the Cascade

The Crash

Since the data pointers were never initialised, the switch tried to read data from a null or corrupted memory address. This caused Switch A to crash and reboot.

The Cascade

Switch A, now crashed, rebooted and immediately sent *its* own flood of new calls (because the network was busy) to *its* neighbours. This triggered the exact same race condition and the exact same bug in them, propagating the failure across the entire network.

The Restoration Challenge

The Problem: Constant Crashing

The network was unstable, with switches constantly crashing and rebooting, triggering further crashes.

The Solution: Coordinated Reboot

The only way to stop the cascade was to turn off all 114 switches simultaneously and bring them back up in a coordinated manner.

The Catch-22

A coordinated reboot required phoning engineers at all 114 locations. But the phone network was down. They couldn't coordinate the fix because the tool needed for coordination was broken.

The Restoration Challenge

The Problem: Constant Crashing

The network was unstable, with switches constantly crashing and rebooting, triggering further crashes.

The Solution: Coordinated Reboot

The only way to stop the cascade was to turn off all 114 switches simultaneously and bring them back up in a coordinated manner.

The Catch-22

A coordinated reboot required phoning engineers at all 114 locations. But the phone network was down. They couldn't coordinate the fix because the tool needed for coordination was broken.

The Restoration Challenge

The Problem: Constant Crashing

The network was unstable, with switches constantly crashing and rebooting, triggering further crashes.

The Solution: Coordinated Reboot

The only way to stop the cascade was to turn off all 114 switches simultaneously and bring them back up in a coordinated manner.

The Catch-22

A coordinated reboot required phoning engineers at all 114 locations. But the phone network was down. They couldn't coordinate the fix because the tool needed for coordination was broken.

The Restoration Challenge

The Solution: Coordinated Reboot

The only way to stop the cascade was to turn off all 114 switches simultaneously and bring them back up in a coordinated manner.

The Catch-22

A coordinated reboot required phoning engineers at all 114 locations. But the phone network was down. They couldn't coordinate the fix because the tool needed for coordination was broken.

The Resolution

The network only recovered nine hours later (11:30 PM) when the call volume naturally dropped low enough to stop the high-load condition that triggered the cascade.

The Lesson: The Limits of Testing

AT&T had extensive testing labs and rigorous procedures. Why was this bug not found before deployment?

The Limitation of Traditional Testing

Their tests never triggered this specific, rare race condition (two messages arriving in rapid succession from a recently rebooted, busy switch).

The Fundamental Question

If you have many tests, how do you know if you have *enough* tests? How do you know what you missed?

The Lesson: The Limits of Testing

AT&T had extensive testing labs and rigorous procedures. Why was this bug not found before deployment?

The Limitation of Traditional Testing

Their tests never triggered this specific, rare race condition (two messages arriving in rapid succession from a recently rebooted, busy switch).

The Fundamental Question

If you have many tests, how do you know if you have *enough* tests? How do you know what you missed?

The Solution: Code Coverage Testing

Code Coverage Testing

A technique (common now, but not in 1990) that instruments the code to track exactly which lines and branches were executed during the test suite run.

The Insight

It reveals which parts of the code have **never** been run.

Application to AT&T

If AT&T had used code coverage, they would have seen that the fatal 'break;' statement was never executed in their test lab.

- This would have prompted them to design a specific test case to trigger that line.
- Running that test would have immediately crashed the switch in the lab, revealing the bug before deployment.

The Solution: Code Coverage Testing

Code Coverage Testing

A technique (common now, but not in 1990) that instruments the code to track exactly which lines and branches were executed during the test suite run.

The Insight

It reveals which parts of the code have **never** been run.

Application to AT&T

If AT&T had used code coverage, they would have seen that the fatal 'break;' statement was never executed in their test lab.

- This would have prompted them to design a specific test case to trigger that line.
- Running that test would have immediately crashed the switch in the lab, revealing the bug before deployment.

The Solution: Code Coverage Testing

Code Coverage Testing

A technique (common now, but not in 1990) that instruments the code to track exactly which lines and branches were executed during the test suite run.

The Insight

It reveals which parts of the code have **never** been run.

Application to AT&T

If AT&T had used code coverage, they would have seen that the fatal 'break;' statement was never executed in their test lab.

- This would have prompted them to design a specific test case to trigger that line.
- Running that test would have immediately crashed the switch in the lab, revealing the bug before deployment.

The Value of Coverage

General Principle

If a line of code has not been run during testing, it is significantly more likely to contain bugs. You should endeavour to test every line and every branch (e.g., both sides of an 'if' statement).

Modern Tools

Modern languages provide tools for this (e.g., 'pytest-cov' for Python).

The Takeaway

Coverage testing is a vital tool for ensuring the robustness of complex software systems, helping to identify untested assumptions and potential failure modes.

The Value of Coverage

General Principle

If a line of code has not been run during testing, it is significantly more likely to contain bugs. You should endeavour to test every line and every branch (e.g., both sides of an 'if' statement).

Modern Tools

Modern languages provide tools for this (e.g., 'pytest-cov' for Python).

The Takeaway

Coverage testing is a vital tool for ensuring the robustness of complex software systems, helping to identify untested assumptions and potential failure modes.

The Value of Coverage

General Principle

If a line of code has not been run during testing, it is significantly more likely to contain bugs. You should endeavour to test every line and every branch (e.g., both sides of an 'if' statement).

Modern Tools

Modern languages provide tools for this (e.g., 'pytest-cov' for Python).

The Takeaway

Coverage testing is a vital tool for ensuring the robustness of complex software systems, helping to identify untested assumptions and potential failure modes.

Conclusion

Conclusion

Summary of Themes

Resilience vs. Perfection (Apollo 11)

- Do not assume software will be bug-free or that hardware will perform as expected.
- Design systems to be resilient, capable of recovery (e.g., restartable tasks) when unforeseen failures occur.

The Danger of Assumptions (Ariane 5)

- Assumptions about the operating environment must be validated. Software reuse requires rigorous retesting in the new context.
- Redundancy must account for systematic software failures, not just random hardware faults. Diversity may be required.

Summary of Themes

Resilience vs. Perfection (Apollo 11)

- Do not assume software will be bug-free or that hardware will perform as expected.
- Design systems to be resilient, capable of recovery (e.g., restartable tasks) when unforeseen failures occur.

The Danger of Assumptions (Ariane 5)

- Assumptions about the operating environment must be validated. Software reuse requires rigorous retesting in the new context.
- Redundancy must account for systematic software failures, not just random hardware faults. Diversity may be required.

Summary of Themes

Complexity and Testing (AT&T)

- Small coding errors, especially those involving complex language semantics (like C's 'break'), can have catastrophic consequences at scale.
- Testing must be comprehensive. Tools like Code Coverage are essential to identify untested code paths and hidden assumptions.